

Animaciones y Transformaciones

Módulo 1: Arquitectura y Maquetación Moderna



Clase 09

Transformaciones y transiciones

¿Qué son las Transformaciones y transiciones?

Transformaciones (transform)

Transiciones (transition)

Animaciones (animation)

Easing: La Curva de Velocidad

animation-fill-mode: Estados de Llegada

will-change

Objetivos de la Clase

Objetivos de la Clase

- Dominar el uso de transition para suavizar cambios de estado (hover, focus).
- Aplicar transformaciones espaciales (rotate, scale, translate) en 2D y 3D.
- Crear animaciones complejas y personalizadas mediante @keyframes y ciclos infinitos.

Transformaciones y transiciones

¿Qué son las Transformaciones y transiciones?

Imagina que diseñar una página web con HTML y CSS es como sacar una foto: todo está estático, congelado y en su lugar. Pero las páginas modernas no son fotos; son interactivas. Se mueven, reaccionan y cambian.

Para lograr que esa "foto" cobre vida, CSS nos da tres herramientas que trabajan juntas para crear movimiento. Podríamos decir que las **transformaciones**, las **transiciones** y las **animaciones** son la trinidad del dinamismo visual en la web.



Transformaciones (transform)

Antes de mover algo, tenemos que definir qué le va a pasar a ese elemento. Para eso sirve la propiedad transform.

Pensá en las transformaciones como "alterar la física" de un objeto de forma instantánea. Podés hacer tres cosas básicas:

Moverlo de lugar (translate).

Cambiar su tamaño (scale).

Girarlo (rotate).



Para que ese cambio se vea suave y natural, usamos las **siguientes herramientas...**

⚠ IMPORTANTE

A tener en cuenta: Por sí sola, una transformación es instantánea y brusca. Si le decís a un botón que gire, va a dar un salto del milisegundo 0 al milisegundo 1. No hay transición.

Transiciones (transition)

Transiciones (transition)

¿Qué son? (Cambios de estado): Requieren un detonante explícito de inicio (ej: un :hover del usuario al pasar el mouse o un cambio de clase mediante JavaScript). Va de un Punto A a un Punto B de forma lineal y directa.

La transición es la encargada de hacer que el cambio no sea un golpe seco. Si el botón pasa de medir 100px a 150px al pasarle el mouse por encima, la transición calcula todos los tamaños intermedios para que el cambio dure, por ejemplo, medio segundo. Es un viaje de ida y vuelta superdirecto.

```
.boton {  
  /* Punto A (Estado Inicial) */  
  background-color: #3498db; /* Azul */  
  padding: 15px 30px;  
  border-radius: 8px;  
  
  /* 1. Definimos la transformación física inicial */  
  transform: translateY(0);  
  
  /* 2. El puente: Decimos qué propiedades queremos suavizar y cuánto van a tardar */  
  transition: transform 0.3s ease;  
}  
  
/* El Detonante: La interacción del usuario */  
.boton:hover {  
  /* Se eleva 5 píxeles hacia arriba */  
  transform: translateY(-5px);  
}
```

Animaciones (animation)

Animaciones (animation)

¿Qué son? (**Control total**): No requieren detonantes externos; pueden iniciarse solas al cargar la página. Permiten trazar líneas de tiempo complejas con múltiples etapas (Punto A -> Punto B -> Punto C -> Punto A) y bucles infinitos.

A diferencia de la transición (que solo va de A a B si el usuario interactúa), la animación es como reproducir un video **autónomo**. Usando algo llamado **@keyframes**, podés decidir exactamente qué hace el elemento en el 10%, en el 50% y en el 100% de su recorrido, permitiendo que un objeto flote de arriba a abajo o cambie de color constantemente en un bucle eterno sin que nadie lo toque.

Para asociar e instrumentar la coreografía del **@keyframes** a un elemento específico:

- **animation-name**: Nombre del @keyframes asociado (ej: deslizar).
- **animation-duration**: Duración de un ciclo completo (ej: 2s).
- **animation-iteration-count**: Cuántas veces se ejecuta: un número fijo o infinite (en bucle eterno).
- **animation-direction**: Dirección del ciclo: normal, reverse o alternate (ida y vuelta constante).
- **animation-play-state**: Permite pausar (paused) o reanudar (running) la animación mediante interacción de JS u hover.

Animaciones (animation)

```
.corazon {
  font-size: 50px;
  text-align: center;
  margin-top: 50px;
  /* Aplicamos la animación:
   - Queremos que dure 1.2 segundos
   - Que se repita de forma infinita
  */
  animation: latido 1.2s infinite ease-in-out;
}

@keyframes latido {
  0% {
    transform: scale(1); /* Estado inicial */
  }
  30% {
    transform: scale(1.25); /* Se agranda rápido */
  }
  60% {
    transform: scale(1); /* Vuelve al tamaño original */
  }
  80% {
    transform: scale(1.15); /* Da un pequeño segundo latido */
  }
  100% {
    transform: scale(1); /* Vuelve al estado inicial */
  }
}
```

Easing: La Curva de Velocidad

Easing: La Curva de Velocidad

Los movimientos en la naturaleza nunca ocurren a velocidad constante lineal (como una máquina). Tienen aceleración e inercia:

- **linear:** Velocidad idéntica de inicio a fin. Se siente robótico y frío.
- **ease-in:** Comienza lento y acelera al final (ideal para elementos que se van de la pantalla).
- **ease-out:** Comienza rápido y desacelera al final (ideal para elementos que entran en pantalla).
- **ease-in-out:** Aceleración inicial y desaceleración final progresiva.

animation-fill-mode: Estados de Llegada

animation-fill-mode: Estados de Llegada

Por defecto, cuando una animación termina, el elemento vuelve bruscamente a sus estilos CSS estáticos originales como si nada hubiese pasado.

- **animation-fill-mode: forwards:** Mantiene los estilos exactos definidos en el fotograma 100% al finalizar la animación de forma permanente.
- **animation-fill-mode: backwards:** Aplica los estilos del fotograma 0% de inmediato antes de que comience el delay de la animación.
- **animation-fill-mode: both:** Combina ambos comportamientos (aplica backwards al inicio y forwards al final).

will-change

will-change

Cuando animamos elementos, el navegador recalcula el renderizado sobre la marcha en tiempo real en la CPU, lo cual puede generar microtironeos de lag en computadoras lentas o dispositivos móviles antiguos.

- will-change: transform, opacity;
- Le avisa al navegador de antemano qué propiedades van a cambiar, permitiendo que la tarjeta gráfica (GPU) reserve memoria dedicada y prepare la aceleración de hardware para lograr una animación súper fluida de 60 FPS.

AVISO

Peligro de Rendimiento: Nunca apliques `will-change` en todos los elementos de tu CSS "por las dudas". Esto consume una cantidad masiva de memoria de la tarjeta gráfica y logrará el efecto contrario, ralentizando por completo el dispositivo del usuario. Usala únicamente en elementos de alto impacto interactivo.



Ejercicio de Clase



Ejercicio de Clase

- Diseña un botón de llamada a la acción .cta-button con estilos profesionales.
- Definí un @keyframes llamado pulse que en el fotograma 50% escale el botón sutilmente a scale(1.08) y cambie el color de sombra para dar un efecto de pulso de atención.
- Aplicá la animación en tu clase .cta-button configurándola para que dure 1.5 segundos, con una curva ease-in-out y que se ejecute de forma indefinida (infinite).
- Añadí una pseudo-clase :hover para que al pasar el mouse por encima de la animación se detenga pausando el ciclo con animation-play-state: paused para permitir que el usuario haga clic de forma cómoda sin que el botón se mueva.

```
.cta-button:hover {  
  animation-play-state: paused;  
}
```

Documentación Oficial de Tecnologías

CSS <https://developer.mozilla.org/es/docs/Web/CSS>

JS <https://developer.mozilla.org/es/docs/Web/JavaScript>

JavaScript <https://developer.mozilla.org/es/docs/Web/JavaScript>

¡Muchas Gracias!

¡Hasta la próxima!

Centro de e-learning